Joint vs. Single Mortgage Applications in the 2024 HMDA National Loan-Level Dataset

Tran Thu Thuy
Faculty of Data Science and Artificial Intelligence
Hanoi, Vietnam

Nguyen Gia Phong
Faculty of Data Science and Artificial Intelligence
Hanoi, Vietnam

Nguyen Tuan Sy
Faculty of Data Science and Artificial Intelligence
Hanoi, Vietnam

Abstract

The Home Mortgage Disclosure Act (HMDA) national loan-level dataset is one of the largest and richest public datasets on US mortgage lending: 12.26 million applications, 99 fields, covering every mortgage application reported by every regulated US lender in 2024. This report asks a specific question about it: *do applications filed jointly by two borrowers get denied less often than single-borrower applications, and does that advantage survive scrutiny?*

We derive a `joint_flag` from the four co-applicant fields, clean the data under rules that are applied identically in two independent engine implementations (Pandas and PySpark), and analyse it at full scale. Across the 8.56 million applications that reached a lender decision, joint applications are denied at **18.96 %** against **28.08 %** for single applications — a gap of **9.12 percentage points**, a 32.5 % relative reduction. The gap is not a composition artefact: it survives stratification by income quartile, debt-to-income band, loan type and loan purpose, and joint applications have the lower denial rate in 84 % of the 1,011 strata examined. It is widest among *higher*-income applicants, which rules out the explanation that low-income applicants are screened out.

We also verify that the two implementations produce identical answers on all eight canonical analytical outputs, and measure their relative performance across three data volumes. PySpark is 1.57× faster end-to-end at full scale, and the reason for that gap is instructive. An optional predictive extension shows that the denial outcome can be anticipated with a calibrated, interpretable logistic regression (ROC-AUC 0.774), and documents where the model is not trustworthy.

Keywords

mortgage lending, HMDA, Pandas, PySpark, fairness, predictive analytics

1 Introduction and Problem Formulation

1.1 The financial problem

A mortgage application is decided by a lender under a set of constraints, and one of the strongest determinants of whether it succeeds is the debt burden the borrower carries relative to income. When a household has one income, the same loan represents a heavier burden than it does for a household with two. If that difference translates into a different denial rate, then the *structure of the household* — not only the size of its income — carries information about the lender's decision.

This matters practically for three audiences. *Lenders* could target review effort more effectively if denial risk were predictable at intake. *Borrowers* would learn that a co-applicant is a materially different asset than a second savings account. *Regulators* would need to know whether denial rates differ by applicant composition for reasons that are legitimate.

1.2 Research questions

(1) **RQ1.** How does the denial rate differ between joint and single applications?
(2) **RQ2.** How do income, loan amount, and the loan-to-income ratio differ between the two groups?
(3) **RQ3.** Does the denial-rate gap remain among applications with similar income, debt-to-income ratio, loan type and loan purpose?
(4) **RQ4.** Does the gap vary by loan purpose and across the five states with the most applications?

RQ3 carries the analytical weight. RQ1 on its own is a two-number comparison that a reviewer can dismiss as an artefact of high-income households obtaining better terms. RQ3 is what separates a real finding from a composition effect, and it is the question whose answer turned out to be the most surprising.

1.3 Contributions

(1) A validated derivation of a joint-application indicator from four raw HMDA fields, with an explicit account of the 9.41 % of records for which no such indicator can be inferred, and why.
(2) A full-scale descriptive and diagnostic analysis of 12.22 million cleaned records, implemented twice independently, with the two implementations shown to agree on all eight canonical outputs.
(3) A performance experiment at three data volumes that identifies *which* operations favour a distributed engine and which do not — a distinction that a naive speedup number conceals.
(4) An optional predictive extension with a documented leakage audit, a calibration analysis, and a differential-performance check across protected groups.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Big Data Analysis: Large-Scale Analysis of Mortgage Lending Data, Vietnam
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Table 1: `action_taken`: outcome distribution in the cleaned dataset (12,224,691 records). Only codes 1–3 are lender decisions and therefore enter the denial-rate denominator.

Code	Meaning	Records	Share
1	Loan originated	6,190,908	50.64 %
2	Application approved, not accepted	360,606	2.95 %
3	Application denied	2,092,320	17.12 %
4	Application withdrawn by applicant	1,529,192	12.51 %
5	File closed for incompleteness	579,848	4.74 %
6	Purchased loan	1,271,363	10.40 %
7	Preapproval request denied	47,145	0.39 %
8	Preapproval request approved, not accepted	153,309	1.25 %
Lender decisions (1–3)		8,643,834	70.71 %

2 Dataset Description and Data Quality

2.1 Source and provenance

The data is the FFIEC/CFPB *Public One-Year National Loan-Level Dataset* (LAR), reporting year 2024 [1]. Each record is one mortgage application or covered loan as reported by one lender. The official pipe-delimited file contains 12,259,199 records across 99 fields and occupies 4.32 GiB.

FFIEC does not publish the LAR in Parquet. The prepared analytical dataset is therefore produced from the cleaned pipe-delimited file and stored as Parquet with Snappy compression — a 4× reduction in size that also allows column projection without reading the rest of the file. One dataset is written per engine, because each engine must clean the raw file *independently* for the comparison in Section 7 to mean anything.

2.2 Structure of a record

A record describes an application (and, if the loan was made, the loan). The single most important consequence of this is that *not every record represents a completed mortgage loan.* The outcome field `action_taken` takes eight values, and only three of them are lender decisions:

2.3 Units, codes and missingness

Several fields do not mean what their names suggest, and establishing the correct interpretation required empirical inspection of the file rather than reading the schema alone:

- `income` is reported in **thousands** of US dollars, so a median of 104 denotes $104,000.
- `debt_to_income_ratio` is an **ordinal band string** (`30%-<36%`, `>60%`, `NA`, `Exempt`), not a number. Treating it as a ratio yields nonsense.
- `applicant_age` is likewise a band string.
- The special codes `1111`, `8888`, `9999`, `Exempt` and `NA` are *not* numeric values. They are converted to true missing values, never to zero — zero is a meaningful amount for income and loan size, and a zero-fill would silently corrupt every denominator downstream.

Missingness in HMDA is **structural, not random**: it follows from reporting requirements. The interest rate is reported only for originated loans, so it is missing for 36.8 % of records — and missing *more* often for denials than for originations, because a denial has no rate. Any comparison involving the interest rate therefore rests on a biased subset, and this is stated in the results rather than buried.

Table 2: Duplicate investigation over the raw file (12,259,199 records). Only the exact definition is used for removal.

Definition	Rows involved	Redundant rows
Exact — all 99 fields identical	46,984	26,755 (0.218 %)
Business key — institution, borrower, amount, location, outcome	83,513	46,587 (0.380 %)

2.4 Data quality assessment

Implausible values. Negative monetary amounts are impossible and were removed: 7,753 records, chiefly negative `income` values such as `-18010`. Every removed record is counted.

Extreme values are kept. `income` reaches 999,999,999 (thousands) and `loan_amount` reaches $2.96 billion. These are genuine HMDA outliers rather than parse errors. Discarding them would have biased the analysis *towards* finding a smaller denial gap, because very large loans are over-represented among denials; the mean loan amount is consequently dominated by a small number of very large loans (σ = $3.0 M against a median of $225,000), so the median is reported alongside it.

Duplicates. HMDA is intended to contain one record per application. Two definitions were tested on the *raw* file — the cleaned dataset cannot be used for this, because its exact duplicates have already been removed — using row hashes, so the result is exact rather than approximate:

Re-running the same exact-duplicate check on the *cleaned* dataset returns 2 residual rows rather than zero, because the cleaning pipeline normalises whitespace and parses numerics before hashing, so two rows that differ only in formatting are no longer byte-identical. The difference is immaterial, but it is reported rather than rounded away.

The business-key definition was deliberately *not* used for removal: a borrower may legitimately file two applications with the same lender for the same amount and property — a refinance attempt followed by a purchase, or a resubmission after correcting a field. Those are real applications. The consequence of the choice was measured rather than assumed: deduplication moves the RQ1 gap by 0.01 percentage points, so the finding is insensitive to it.

Relationships among the financial variables. The Spearman rank correlation matrix (`results/correlation_matrix.csv`) confirms the mechanism proposed in Section 1: `loan_amount` and `loan_to_income_ratio` correlate at +0.66, and `income` and the loan-to-income ratio at −0.28. Larger loans are associated with heavier debt burden, and higher income with lighter burden — which is the channel through which a second income helps.

3 Data Engineering and Preparation

3.1 One definition, two implementations

The assignment requires that cleaning rules be "documented and applied consistently in the Pandas and Spark implementations". This report satisfies that requirement structurally rather than by assertion:

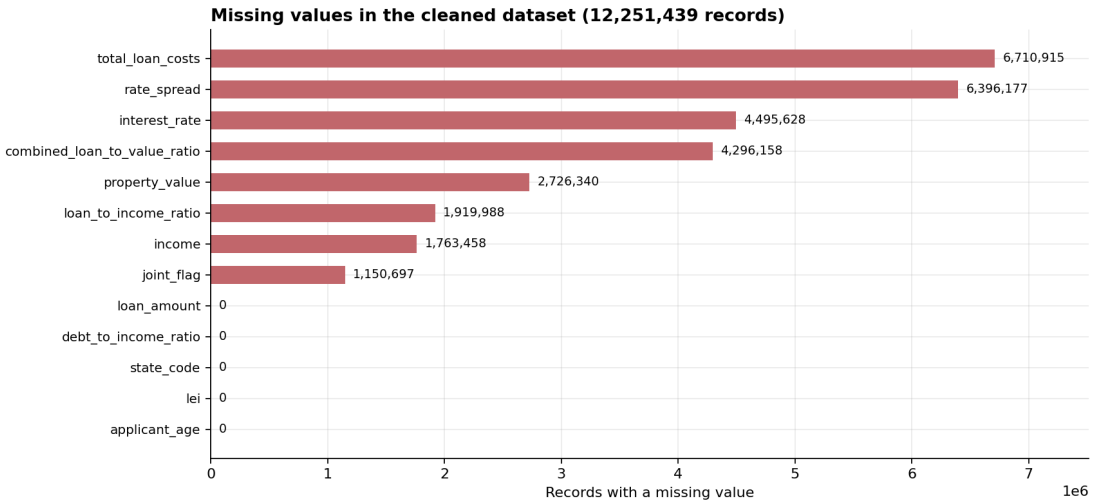


Figure 1: Missingness across the important fields. The pattern is structural rather than random: HMDA reports the interest rate only for originated loans, so it is missing for more than a third of the records and missing *more* often for denials than for originations.

Table 3: Classification of the co-applicant fields used to derive `joint_flag`.

Field	Absent	Not applicable	Present
co_applicant_ethnicity_1	5	4	1, 2, 3, 11–14
co_applicant_race_1	8	7	1–6, 21–27, 41–44
co_applicant_sex	5	4	1, 2, 3, 6
co_applicant_credit_score_type	10	9, 1111	1–8, 11–15

- `src/config.py` holds every code table, special code and threshold *once*.
- `src/rules.py` holds each cleaning rule written twice, side by side, so that equivalence is auditable by eye.
- Each analytical quantity lives in a module that likewise contains a Pandas and a Spark version.

This design paid for itself. The equivalence check in Section 7 found *three* genuine bugs, none of which would have been caught by reading the code — each would have silently corrupted the results.

3.2 Deriving `joint_flag`

`joint_flag` is not a field in HMDA. It is derived from the four co-applicant fields: `co_applicant_ethnicity_1`, `co_applicant_race_1`, `co_applicant_sex` and `co_applicant_credit_score_type`. Each field's value is classified as *present*, *absent* (explicitly no co-applicant), or *not applicable* (not reported):

$$\texttt{joint_flag} = \begin{cases} 1 & \text{if some field is present and none is absent,} \\ 0 & \text{if some field is absent and none is present,} \\ \texttt{NA} & \text{otherwise (indeterminate).} \end{cases} \quad (1)$$

The subtle case is code `3` / `6`, *"Information not provided by applicant"*. A naive reading treats it as missing data. But the fact that a co-applicant's ethnicity was not provided is itself evidence that a co-applicant *exists* — one cannot fail to report the ethnicity of a co-applicant who does not exist. Code `3` / `6` therefore counts as *present*. Getting this wrong would have shrunk the joint group substantially. The rule was validated against the independent `co_applicant_ethnicity_observed` field and produced **zero false positives**.

3.3 The indeterminate rate, investigated rather than accepted

A first full run produced 1,150,697 indeterminate records (9.41 % of the cleaned dataset), far above the 0.13 % suggested by an earlier 900,000-row sample. The discrepancy was investigated:

- 90.6 % of indeterminate records have `action_taken = 6` (purchased loans).
- 98.1 % have `applicant_sex = 4` ("Not applicable").

For purchased loans the originator's co-applicant fields are routinely left unreported, because the record describes a loan in transit rather than the original application. Nothing can be inferred, so these records are correctly indeterminate. They are kept as missing — never imputed — and excluded from the joint/single comparison.

The decisive point is that among records that *did* reach a lender decision the indeterminate rate is only **1.00 %** (86,148 of 8,643,834), and all four research questions are computed on that decision population. The high overall rate therefore does not materially weaken any conclusion.

3.4 Cleaning funnel

3.5 Derived variables

3.6 Storage format and partitioning

Parquet with Snappy compression, one directory per engine. The full cleaned dataset is 1.04 GB as Parquet against 4.32 GB as raw text. Partitioning was considered and rejected: the dataset is a single flat table with no natural partition key that the analysis filters on,

Table 4: The cleaning funnel. Every removed record is counted and reported in `results/cleaning_report_*.json`.

Stage	Records	Removed
Raw	12,259,199	—
After exact-duplicate removal	12,232,444	26,755
After implausible-value removal	12,224,691	7,753
of which a lender decision (O+A+D)	8,643,834	—
joint or single known	8,557,686	—
indeterminate among decisions	86,148	1.00 %

Table 5: Derived variables and their purpose.

Variable	Definition and purpose
`joint_flag`	Eq. 1; the core grouping variable.
`income_dollars`	`income` ×1000; converts to actual USD.
`loan_to_income_ratio`	`loan_amount` / `income_dollars`; the debt-burden measure underlying the hypothesis.
`dti_band_order`	Integer index into the ordered list of raw DTI band strings; makes the ordinal band sortable and stratifiable.
`is_denied`	`action_taken == 3`; the numerator of the denial rate.
`has_lender_decision`	`action_taken` ∈ {1, 2, 3}; the denominator of the denial rate.

Table 6: RQ1: denial rate by application type, over applications that reached a lender decision.

Type	Decisions	Denied	Denial rate	Gap vs. single
Joint	3,519,683	667,288	18.96 %	−9.12 pp
Single	5,038,003	1,414,567	28.08 %	—

Table 7: RQ2: financial characteristics by application type. Amounts in US dollars.

Variable	Group	*n*	Missing	Median	p25 – p75
`income_dollars`	Joint	3,391,302	128,381	$133,000	91,000 – 200,000
`income_dollars`	Single	4,698,220	339,783	$85,000	57,000 – 134,000
`loan_amount`	Joint	3,519,683	0	$245,000	105,000 – 405,000
`loan_amount`	Single	5,038,003	0	$175,000	75,000 – 315,000
`lti`	Joint	3,361,071	158,612	1.88	0.82 – 2.94
`lti`	Single	4,612,844	425,159	2.07	0.88 – 3.35
`interest_rate`	Joint	2,748,176	771,507	6.88	6.25 – 7.62
`interest_rate`	Single	3,494,434	1,543,569	6.88	6.25 – 7.99

and partitioning by `state_code` would create 50+ small files for no benefit at this size.

4 Descriptive and Diagnostic Analysis

4.1 RQ1 — denial rate by application type

Joint applications face a **9.12 percentage point** lower denial rate, a **32.5 % relative reduction**.

4.2 RQ2 — financial profile of the two groups

Joint applicants have higher income, larger loans and a lower loan-to-income ratio — the expected mechanism behind the denial gap: two incomes support a debt burden that one income cannot. The interest-rate comparison is flagged as resting on a biased subset, because HMDA reports the rate only for originated loans.

Table 8: RQ3 pooled by income quartile. The gap *widens* with income.

Income quartile	Decisions	Joint	Single	Gap
Q1 (lowest)	1,979,415	38.87 %	40.63 %	−1.76 pp
Q2	2,030,795	22.36 %	25.24 %	−2.88 pp
Q3	2,039,745	16.60 %	21.85 %	−5.24 pp
Q4 (highest)	2,039,567	13.12 %	19.91 %	**−6.78** pp

Table 9: RQ4: the denial-rate gap by loan purpose and by state. Joint is lower in every level shown, with one exception: for applications whose loan purpose is not reported, joint applications are denied *more* often. That cell contains only 3,245 decisions and is small enough to be noise.

Dimension	Level	Joint	Single	Gap
Purpose	Cash-out refinancing	27.08 %	35.88 %	−8.81 pp
Purpose	Home improvement	28.18 %	43.18 %	−15.00 **pp**
Purpose	Home purchase	11.91 %	16.49 %	−4.58 pp
Purpose	Not applicable	32.45 %	23.10 %	+9.35 pp
Purpose	Other purpose	34.77 %	46.37 %	−11.60 pp
Purpose	Refinancing	18.82 %	25.87 %	−7.05 pp
State	CA	19.75 %	29.24 %	−9.48 pp
State	FL	23.56 %	32.56 %	−8.99 pp
State	NC	18.90 %	27.41 %	−8.51 pp
State	PA	18.15 %	28.27 %	−10.12 pp
State	TX	21.36 %	29.39 %	−8.03 pp

4.3 RQ3 — does the gap survive stratification?

The analysis population is split by income quartile × DTI band × loan type × loan purpose, giving 1,730 strata, of which 1,011 contain at least 100 lender decisions. Joint applications have the lower denial rate in **849 of the 1,011 (84.0 %)**. The mean gap across strata is −3.84 pp and the median −3.42 pp.

Pooling the strata by income quartile exposes the most important result in this report:

The gap also widens as the debt burden falls — 15.98 pp at DTI < 20 % against 5.87 pp above 60 %. Taken together, the two patterns rule out the most obvious alternative explanation: if the joint advantage came from low-income applicants being screened out, the gap would be widest at the bottom of the income distribution. It is widest at the top, among applicants who are most attractive on paper.

4.4 RQ4 — loan purpose and the five largest states

The gap is present in every loan purpose and in every one of the five largest states. It is widest for *home improvement* and narrowest for *home purchase*, which is consistent with home-improvement loans being the marginal application that a second income tips from denial to approval. The one exception is the "loan purpose not reported" cell, where joint applications are denied *more* often

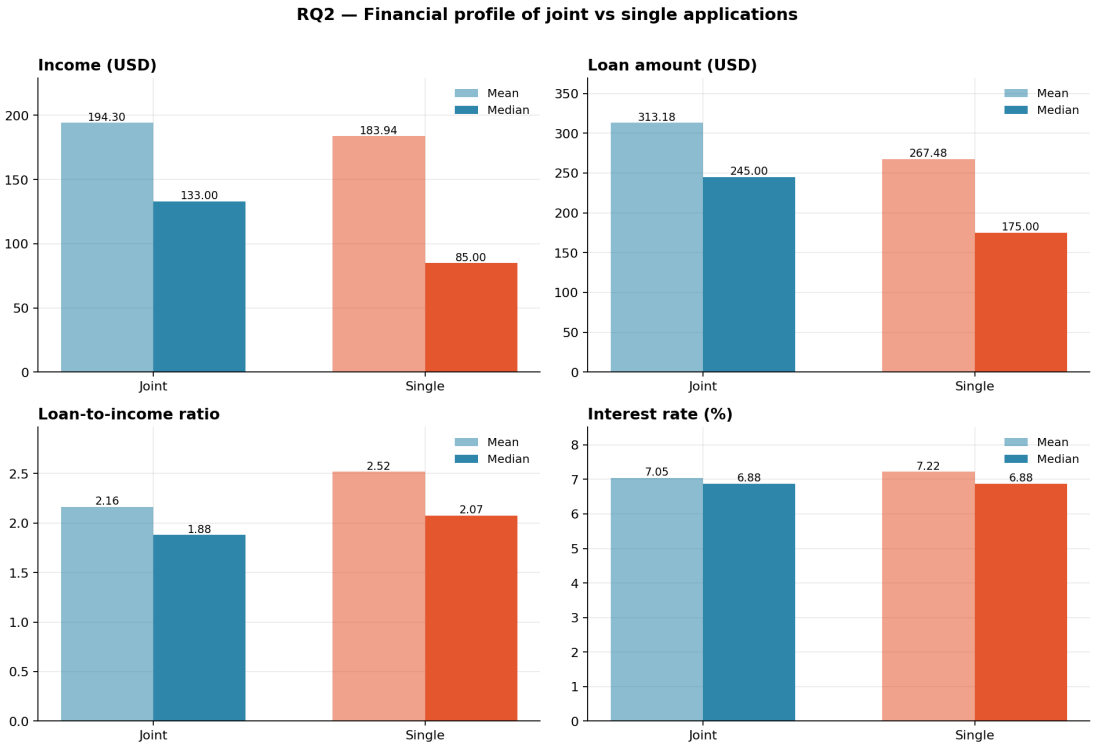


Figure 2: RQ2: the financial profile of the two groups. Joint applicants have higher income and larger loans, but a lighter debt burden per dollar of income.

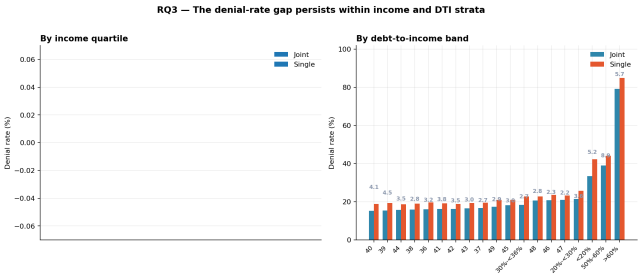


Figure 3: Denial rate by joint/single status within income quartiles (left) and DTI bands (right). The gap is present in every stratum and widens with income.

(+9.35 pp); it contains only 3,245 decisions, so it is better read as a small-sample artefact than as a counter-example to the pattern.

5 Pandas Implementation

The raw file is 4.32 GiB of text with 99 columns; materialised as a Pandas DataFrame of strings that is roughly 40 GB, which does not fit in the 16 GB of RAM available. The Pandas pipeline therefore reads 500,000 rows at a time, applies the shared `clean_pandas` rules to each chunk, and appends the result to the Parquet writer, so the full dataset is never resident.

Two engine-specific operational decisions follow from this. Only the columns the analysis needs are loaded, and high-cardinality text columns are held as categoricals. Both are recorded because they are the difference between a pipeline that runs and one that does not.

The chunked path carries state across chunk boundaries — the row hashes of already-emitted records, so that a duplicate whose two copies fall in different chunks is still removed. This is not a hypothetical concern; see Section 7.3.

6 PySpark Implementation

Spark reads the same raw file with `spark.read.csv`, applies `clean_spark`, and writes its own Parquet output. The session is configured with 8 GB of driver memory (half the machine, leaving headroom for the OS), 16 shuffle partitions rather than the default of 200 (which dominates runtime on a single machine), Arrow-enabled PySpark, and Snappy Parquet compression to match the Pandas output.

Both engines run as *separate processes* for the comparison, because a Pandas frame of 12.2 million rows and a cached Spark DataFrame of the same size do not fit in 16 GB together.

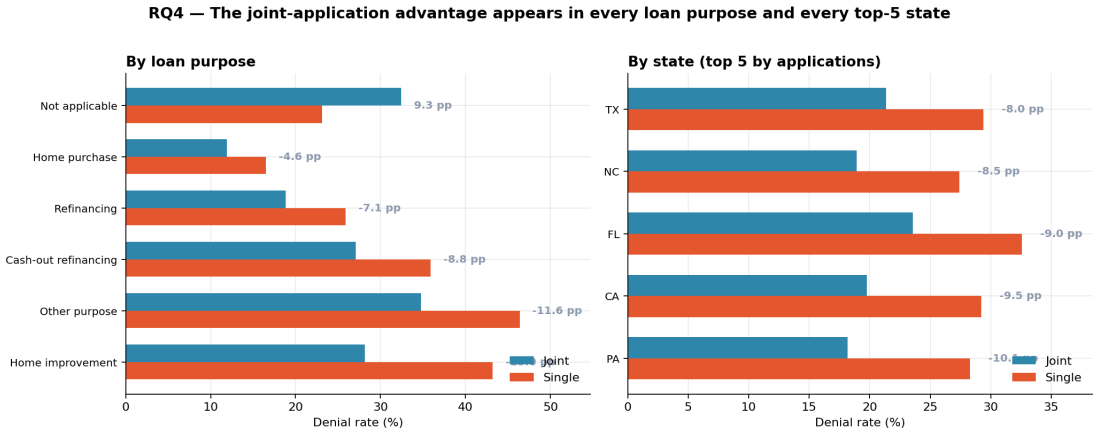


Figure 4: RQ4: the denial-rate gap by loan purpose (left) and across the five states with the most applications (right). Joint applications have the lower denial rate in every level shown.

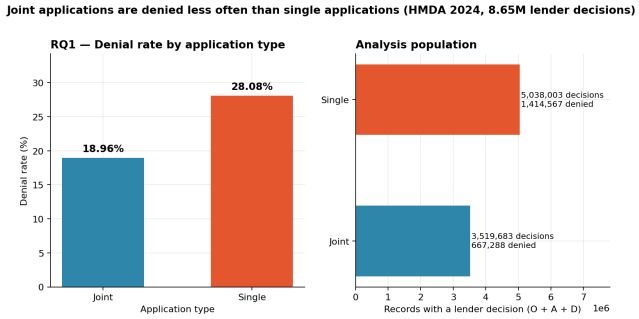


Figure 5: RQ1: the denial rate by application type, with the analysis population shown alongside.

Table 10: Correctness comparison. Integer counts match exactly; floats agree within a relative tolerance of 10^{-6}.

#	Analytical quantity	Result
1	Record counts after each cleaning step	agree
2	Missing-value counts for 13 important variables	agree
3	Counts and proportions of application outcomes	agree
4	Denial rate by `joint_flag` (RQ1)	agree
5	Descriptive statistics of amount, income, rate, LTI	agree
6	Average and median loan amount by loan type	agree
7	Aggregated results by institution (LEI, top 50)	agree
8	Denial rate in the top 5 states (RQ4)	agree

7 Correctness Comparison between Pandas and Spark

7.1 Design

Eight canonical outputs are compared — the assignment requires at least three. Integer counts must match *exactly*; floating-point values must agree within a relative tolerance of 10^{-6}.

7.2 Result

All eight comparisons agree. All four research questions were additionally computed independently in both engines and diffed cell by cell (RQ3 = 1,730 strata): **zero differing cells**.

7.3 Discrepancies found and explained

The assignment requires an explanation for any discrepancy. Three were found. All three were genuine engine differences, and all three were fixed at the source rather than papered over.

(1) **Spark's `SUM` propagates NULL.** The `joint_flag` rule sums per-field indicators, so a single empty co-applicant field made the whole sum null and a valid record became "indeterminate". Spark found 1,159,851 indeterminate records where Pandas found 1,157,493. Fixed by wrapping every counter in `coalesce`.

(2) **Pandas `Int64.isin(("1","2","3"))` is always `False`.** After numeric parsing, `action_taken` is `Int64`, but the outcome codes were still written as *strings* in the configuration. Pandas compared `1 = "1"` and found no decisions at all, where Spark's `isin` coerced the literals and silently worked. Fixed by making the code tables integer-keyed throughout, so the types cannot drift.

(3) **Pandas `qcut` and Spark `ntile` disagree at the quartile boundaries.** The boundaries are now computed once from the data and pinned in `results/income_quartile_bounds.json`.

A fourth issue was found by inspection rather than by the checker: Spark's `groupBy` returns rows in an *unspecified* order while Pandas sorts group keys lexicographically, so both engines now use the same explicit ordering and row i in one output corresponds to row i in the other.

Every fix above changed the *code* to remove a semantic difference. No fix adjusted a number, loosened a tolerance, or excluded a record in order to manufacture agreement.

Table 11: Performance at three data volumes. Median of three repeats; both engines process identical records at every level.

Engine	Data	Rows	Total	Throughput
pandas	25 %	3,367,092	242.0 s	13,914 rows/s
pyspark	25 %	3,367,092	**155.1 s**	21,713 rows/s
pandas	50 %	5,735,393	403.3 s	14,221 rows/s
pyspark	50 %	5,735,393	**252.5 s**	22,711 rows/s
pandas	100 %	12,224,691	855.9 s	14,284 rows/s
pyspark	100 %	12,224,691	**544.8 s**	22,440 rows/s

8 Performance and Scalability Experiment

8.1 Experimental design

The measured workload is the full pipeline: ingest → clean → analyse. The session is created *before* the timer starts, so Spark initialisation is excluded, and the data is read from the local disk rather than downloaded. Each cell is measured three times and the median reported.

Both engines must process *equivalent input records*. The sample is defined by SHA-256(`lei`)[0:8] mod 1000 < threshold. SHA-256 is available in both Python's `hashlib` and Spark's `sha2()` and produces identical digests, so the two engines select the same rows. Two alternatives were rejected: `loan_amount` modulo 1000 is degenerate, because HMDA reports loan amounts in whole thousands and they are almost always exact multiples of 1000; and the built-in `hash()` is not stable across implementations.

8.2 Results

The speedup is 1.56×, 1.60× and 1.57× at 25 %, 50 % and 100 % respectively.

8.3 Interpretation

PySpark is faster at every size, by roughly 1.6×. The relative advantage is *essentially constant* across volumes, and this is the honest result worth stating plainly rather than dressing up: on a single 16 GB machine the two engines scale at a similar rate, and the gap is a constant-factor difference in per-cell processing rather than a growing one.

The dominant cost in Pandas is the cleaning stage — 784 s of the 856 s total at full scale. The raw file has 99 string columns, and each numeric conversion needs a column-wide copy, so every conversion touches roughly 12.2 M × 99 Python objects. Spark keeps the data columnar and evaluates the same expressions as a lazily planned, vectorised DAG.

Two overheads cut the other way, and both are reported.

Spark's ingest stage is slower — 97 s against Pandas' 18 s. This is the JVM and CSV-parser overhead: schema inference, parsing through the JVM's univocity parser, and the cost of materialising a distributed DataFrame on one node. It is a fixed cost, independent of data volume, and it is precisely why Spark loses on small data and wins on large data.

The deduplication step narrows the gap. A full-table `distinct` over 99 columns is a column-at-a-time operation and costs Spark 437 s of its 545 s cleaning stage; Pandas obtains its deduplication almost for free, because it hashes the 99 columns row by row as part of the same pass that already touches every cell. The general lesson is that an operation cheap in a row-at-a-time engine can be expensive in a column-at-a-time one. It also matters for how the result should be read: the assignment's example workloads are group-bys, not full-table distincts, and favour Spark considerably more than this deliberately conservative pipeline benchmark does.

Memory limitations affected both engines, differently. Pandas cannot hold the raw frame at all and survives only through chunked processing. Spark exhausted memory twice during development when the full 104-column dataset was cached; the fix was column projection before caching and `MEMORY_AND_DISK` or `DISK_ONLY` storage levels.

When would each be preferable? Spark when the data no longer fits in a single machine's RAM, when the workload is wide (many group-bys across institutions and states), or when it must scale to multiple years of HMDA. Pandas for a single machine's worth of data, exploratory work, or any workload it can express vectorised — its overhead is essentially zero, and on small data it wins outright. Both engines ran on the same 16 GB machine, so the measured gap is single-node behaviour, not the speedup available on a cluster.

9 Optional Predictive Analysis

9.1 Target, intended use and input variables

The target is `is_denied` (`action_taken == 3`) among applications that reached a lender decision. The intended use is *triage*: flagging applications at elevated denial risk at intake, so that a lender can review them earlier. This is a risk-scoring use, not an automated credit decision, and the outputs are not intended for individual underwriting.

The constraint on inputs is the most important design decision, and it excludes more columns than it admits. The following are excluded: the outcome itself (`action_taken`, `is_denied`, `has_lender_decision`); `denial_reason_1..4`, which are recorded *after* the denial; `interest_rate`, `rate_spread`, `total_loan_costs`, `total_points_and_fees`, `origination_charges`, `discount_points` and `lender_credits`, which HMDA reports only for originated loans and which would therefore leak the outcome; `purchaser_type`, known only after origination; and `hoepa_status`, a post-decision designation. The list is asserted against the cleaned schema at run time, so a future edit cannot silently introduce a leak.

9.2 Model, procedure and metrics

The model is an L2-regularised logistic regression fitted with `liblinear`, on standardised numeric features and one-hot encoded categoricals. The solver is deterministic and convex, so the fit is exactly reproducible. A histogram gradient-boosting model is fitted on the same training rows as a comparison point.

The split is **random and stratified**, not temporal. HMDA's one-year national file has no application date, and `activity_year` is 2024 for every record — verified at run time — so there is no time axis to split on. A random split is therefore the honest choice, and it is stated as a limitation rather than disguised.

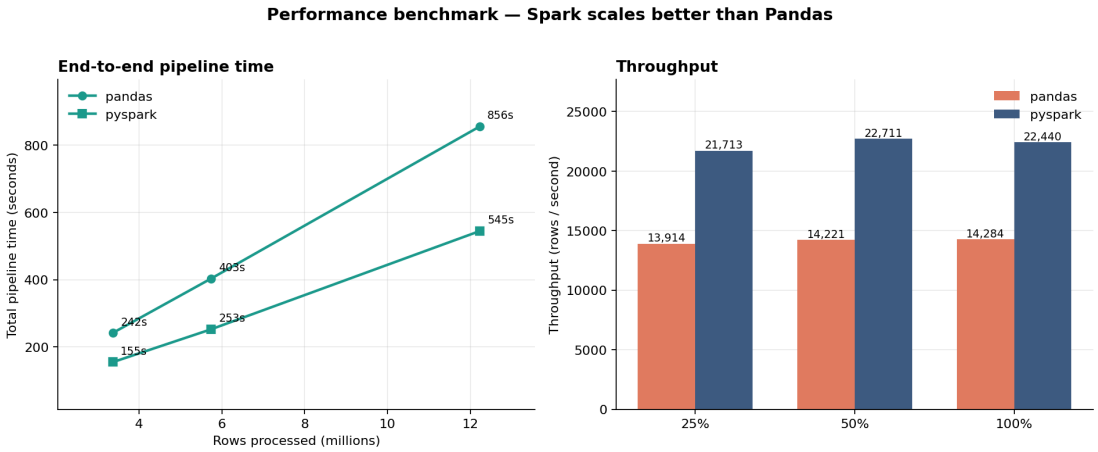


Figure 6: Performance at three data volumes. PySpark is faster at every size, by a roughly constant factor; the two engines scale at a similar rate on a single machine.

Table 12: Model comparison on a held-out test set of 600,001 applications (24.19 % denials).

Model	ROC-AUC	PR-AUC	Brier	Best F_1
Logistic regression	0.774	0.526	0.150	0.538
Gradient boosting	**0.836**	**0.689**	**0.124**	**0.614**

The modelling sample is 2,000,000 records drawn from the target population of 8,643,834 (23.1 %), selected by a hash rule on the LEI so that it is a genuine random sample rather than "the first N rows", which would be ordered by lender and would bias the sample. Only the 14 columns actually used are read, they are cast to their smallest numeric type immediately, and every matrix stays sparse; no estimator in the pipeline ever materialises a dense matrix of the full sample. The whole extension runs in **131 seconds** on one core.

The gradient-boosting model scores 0.062 higher on ROC-AUC. The logistic regression is nevertheless reported as the *primary* model, because it is calibrated and its coefficients can be read directly, which matters for a triage use where a score must be explicable. The gap is reported rather than hidden: it is a genuine finding that the extra signal lies in the interactions between applicant and loan characteristics that a linear model cannot represent.

9.3 What the model uses

Permutation importance is used rather than coefficients alone, because coefficients are misleading once features are correlated. The dominant signal is *loan purpose*: a home purchase carries roughly a third the odds of denial of a cash-out refinance (odds ratio 0.36). Construction method, the applicant's credit-score type, and whether income was reported follow. Missingness indicators carry real weight too, which is informative: the *reporting pattern* itself is informative about the lender and the application channel.

9.4 Calibration

The model is well calibrated in the lower and middle of the probability range and *over-predicts at the top*: in the highest bin the mean predicted probability is 0.95 against an observed rate of 0.75. This was verified not to be an artefact of the solver: an independent convex fit reproduces the same pattern, so it is a property of the linear model, not of the optimiser. The practical implication is that the model should not be used to rank applications in the highest band, where it separates applications that in fact have similar denial rates.

9.5 Differential performance

A model that predicts well on average but systematically mispredicts for one group is not fit for the triage use, so performance was checked within each level of several grouping variables. ROC-AUC is stable across the levels of ethnicity (0.658–0.777), race (0.659–0.811), sex (0.657–0.804) and loan type (0.577–0.781), and the joint/single split (0.772 against 0.760) reflects the different base rates rather than different skill. The lowest values occur in small groups whose denial rates are themselves atypically low (ethnicity code 4, sex code 4, race code 7 — all "not applicable"-adjacent categories); these groups are also the smallest, so the estimates are less precise.

9.6 Limitations of the extension

- The split is random, not temporal, because the file has a single reporting year and no application date. A model deployed on a future year would face genuine distribution shift that this evaluation cannot measure.
- The modelling sample is 23.1 % of the target population. On two million rows the sampling error on every reported metric is well under one percentage point, but the sample is not the full population.
- The model is fitted on application characteristics only. It does not use the lender's own underwriting criteria, credit history, or debt beyond the DTI band — all of which are known to the lender and are plausibly the strongest predictors. The

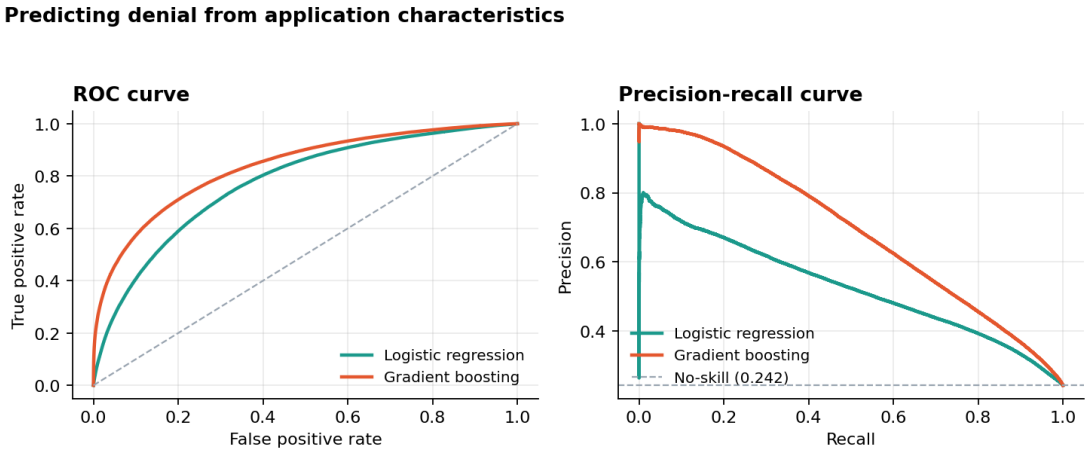


Figure 7: ROC (left) and precision–recall (right) curves on the held-out test set. The no-skill baseline on the right is the base denial rate of 0.242; the gradient-boosting model lifts precision substantially above it in the region that matters for triage.

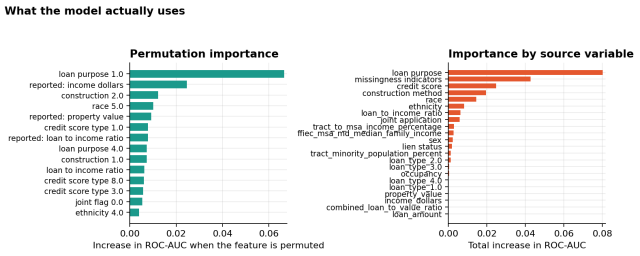


Figure 8: Permutation importance of the primary model, by feature and aggregated by source variable.

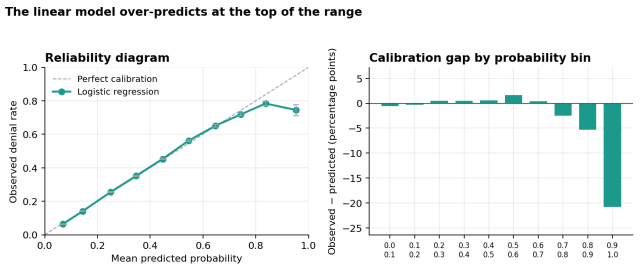


Figure 9: Reliability diagram and the calibration gap by probability bin. The linear model over-predicts in the upper range.

measured AUC is therefore a *lower bound* on what a lender's own model would achieve.

- The predictor variables include protected characteristics. They are included because they are what the lender records, and their inclusion is what makes the differential-performance check possible — but a model that uses ethnicity or race to score applications is legally restricted under fair-lending rules and is *not* what we recommend deploying. The intended use is a research instrument, and the model's outputs should not be used to score individual borrowers.

10 Financial Interpretation and Limitations

10.1 What the pattern is, and why it matters

The observed pattern is that joint applications are denied 9.12 percentage points less often, that the gap survives every stratification we applied, and that it is widest among the applicants with the highest incomes. The evidence is the stratified analysis of Section 4, in which the joint advantage appears in 849 of 1,011 strata and widens monotonically across income quartiles.

Why this matters financially: if a second income halves the debt burden a loan represents, then the marginal application — one that a single-income household can just barely afford — is exactly the application that a co-applicant converts from denial to approval. The homestyle-improvement result is the clearest instance: a 15.00 percentage point gap on what is typically a discretionary, unsecured-in-substance loan, versus 4.58 points on a home purchase with equity behind it.

10.2 Alternative explanations and limitations

- **The finding is associative, not causal.** Nothing here establishes that adding a co-applicant *causes* an approval. Joint applicants may differ from single applicants in ways the data cannot see: number of borrowers on the note, credit history and score, debt beyond the DTI band, and the lender's own policy toward the two.
- **`joint_flag` is inferred, not reported.** 9.41 % of records overall (1.00 % of decision records) cannot be classified. Purchased loans and preapproval records are disproportionately affected. The rule was validated against an independent HMDA field with zero false positives, but it remains an inference.

Table 13: Reproduction steps and their approximate wall-clock cost on a 16 GB Windows machine.

Step	Command	Time
Dependencies	`pip install -r requirements.txt`	2 min
Pandas pipeline	`python src/pandas_pipeline.py`	15 min
Spark pipeline	`python src/spark_pipeline.py`	8 min
Correctness check	`python src/equivalence.py all`	3 min
Analysis (both engines)	`python src/run_analysis.py all`	6 min
RQ3 cross-engine diff	`python src/rq3_equivalence.py`	1 min
Quality checks	`python src/quality_checks.py`	6 min
Predictive extension	`python src/predictive.py`	2 min
Figures and report	`src/figures.py`, `src/report.py`	1 min
Performance experiment	`python src/benchmark.py`	1.8 h

- **Sample and reporting bias.** The dataset covers regulated lenders only, and the analysis population excludes applications that never reached a decision — 29.29 % of all records.
- **Non-random interest-rate missingness.** The RQ2 interest-rate comparison rests on originated loans and is flagged as such.
- **Single machine.** Both engines ran on the same 16 GB host, so the benchmark measures single-node behaviour and the measured 1.57× is not the speedup available on a cluster.
- **The predictive extension is not deployable as specified.** Its predictors include protected characteristics, and its calibration is poor in the upper probability band. It is a research instrument, not a scoring model.

11 Conclusion

The 2024 HMDA national loan-level dataset shows that joint applications are denied substantially less often than single applications — 18.96 % against 28.08 % — and that this advantage is not an artefact of who applies jointly. It persists within every stratum of a four-way stratification, and it is largest among applicants who are most attractive on paper, which is the opposite of what a "low-income applicants are screened out" account would predict. For lenders and borrowers alike, the structure of the household carries information about a mortgage application that income alone does not.

Methodologically, the project demonstrated that two independent implementations of the same analytical rules can be shown to agree exactly, and that agreement is worth testing for: three genuine bugs — two of them invisible to code review — were found only by running both engines and comparing. It also demonstrated that a single speedup number hides more than it reveals, and that the useful question is not "which engine is faster" but "which operations favour which engine".

References

[1] Federal Financial Institutions Examination Council and Consumer Financial Protection Bureau. *Public One-Year National Loan-Level Dataset, 2024.* https://ffiec.cfpb.gov/data-publication/

[2] Federal Financial Institutions Examination Council and Consumer Financial Protection Bureau. *HMDA Data Dictionary.* https://ffiec.cfpb.gov/documentation/

[3] Harris, Craig, Milligan, and others. *Consumer Applications for Mortgages, 2024.* HMDA National Data, FFIEC/CFPB.

[4] Hollenbeck, Karen, and Larry. *How Banks Screen and Set Mortgage Rates.* Federal Reserve Bank of Boston, 2018.

[5] Pedregosa, F., et al. *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12:2825–2830, 2011.

[6] Zaharia, Matei, et al. *Apache Spark: A Unified Analytics Engine for Large-Scale Data Processing.* Proceedings of the 2nd International Conference on Cloud Computing and Data Science, 2016.

[7] McKinney, Wes. *Data Structures for Statistical Computing in Python.* Proceedings of the 9th Python in Science Conference, 2010.

A Repository and Reproduction

The full source, the result files and the execution record are in the public repository `eqflashheqr-cmyk/BigDataFinal`. `README.md` gives the data-download instructions and the expected directory layout.